



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

Scuola  
di Ingegneria

Corso di Laurea in  
Ingegneria Informatica

Raccolta dati e creazione di un dataset di  
traiettorie di guida per l'addestramento di  
moduli di guida autonoma

Data collection and creation of a driving  
trajectory dataset for training  
autonomous driving modules

**Candidato:**  
Francesco Taccetti

**Relatore:**  
Pietro Pala

**Correlatori:**  
Lorenzo Seidenari  
Federico Becattini

# Indice

|                                                                  |            |
|------------------------------------------------------------------|------------|
| <b>Elenco delle figure</b>                                       | <b>iii</b> |
| <b>Introduzione</b>                                              | <b>iv</b>  |
| <b>1 Descrizione del modello AutoRC</b>                          | <b>1</b>   |
| 1.1 Specifiche del modello e dispositivi equipaggiati . . . . .  | 1          |
| 1.1.1 Telecamera ZED Stereolabs 2i . . . . .                     | 2          |
| 1.2 Infrastruttura del sistema . . . . .                         | 3          |
| 1.2.1 Blocco MUX . . . . .                                       | 3          |
| 1.2.2 Blocco controllore . . . . .                               | 3          |
| 1.2.3 Blocco spostamento e assetto . . . . .                     | 4          |
| 1.3 Salvataggio messaggi attraverso il framework ROS2 . . . . .  | 4          |
| <b>2 Raccolta ed elaborazione dei dati</b>                       | <b>6</b>   |
| 2.1 Procedura di acquisizione dei dati . . . . .                 | 6          |
| 2.2 Estrazione dei campioni dalle registrazioni . . . . .        | 7          |
| 2.2.1 Struttura delle classi del modulo software . . . . .       | 7          |
| 2.2.2 Il metodo <i>get_sample()</i> . . . . .                    | 8          |
| 2.3 Visualizzazione dei campioni . . . . .                       | 9          |
| <b>3 Analisi statistiche dei dati raccolti</b>                   | <b>14</b>  |
| 3.1 Statistiche generali delle sequenze . . . . .                | 14         |
| 3.1.1 Angolo di curvatura iniziale . . . . .                     | 14         |
| 3.2 Dataset 1: $\delta_f = 2$ . . . . .                          | 15         |
| 3.2.1 Trattorie del dataset . . . . .                            | 15         |
| 3.2.2 Angolo di curvatura . . . . .                              | 15         |
| 3.2.3 Lunghezza delle traiettorie . . . . .                      | 16         |
| 3.2.4 Distanza fra punti consecutivi delle traiettorie . . . . . | 18         |
| 3.2.5 Framerate . . . . .                                        | 18         |
| 3.2.6 Area dell'immagine RGB coperta dalla point-cloud . . . . . | 18         |
| 3.3 Dataset 2: $\delta_f = 4$ . . . . .                          | 20         |
| 3.3.1 Trattorie del dataset . . . . .                            | 20         |
| 3.3.2 Angolo di curvatura . . . . .                              | 20         |
| 3.3.3 Lunghezza delle traiettorie . . . . .                      | 21         |
| 3.4 Dataset 3: $\delta_f = 6$ . . . . .                          | 22         |

|                                             |           |
|---------------------------------------------|-----------|
| <i>INDICE</i>                               | ii        |
| 3.4.1 Trattorie del dataset . . . . .       | 22        |
| 3.4.2 Angolo di curvatura . . . . .         | 22        |
| 3.4.3 Lunghezza delle traiettorie . . . . . | 22        |
| <b>Conclusioni</b>                          | <b>24</b> |
| <b>Bibliografia</b>                         | <b>24</b> |

# Elenco delle figure

|      |                                                                                                 |    |
|------|-------------------------------------------------------------------------------------------------|----|
| 1.1  | Il modello AutoRC . . . . .                                                                     | 1  |
| 1.2  | Schema a blocchi rappresentante l'infrastruttura del sistema . . . . .                          | 3  |
| 2.1  | Diagramma delle classi del modulo software . . . . .                                            | 7  |
| 2.2  | Sistema di riferimento della fotocamera ZED . . . . .                                           | 9  |
| 2.3  | Visualizzazione della nuvola di punti e della traiettoria- Campione 1 . . . . .                 | 10 |
| 2.4  | Visualizzazione della nuvola di punti e della traiettoria - Campione 2 . . . . .                | 10 |
| 2.5  | Visualizzazione della nuvola di punti e della traiettoria - Campione 3 . . . . .                | 11 |
| 2.6  | Visualizzazione della nuvola di punti e della traiettoria - Campione 4 . . . . .                | 11 |
| 2.7  | Visualizzazione della nuvola di punti e della traiettoria - Campione 5 . . . . .                | 12 |
| 2.8  | Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 1 . . . . .               | 12 |
| 2.9  | Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 2 . . . . .               | 13 |
| 2.10 | Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 3 . . . . .               | 13 |
| 2.11 | Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 4 . . . . .               | 13 |
| 3.1  | Distribuzione dell'angolo d'attacco . . . . .                                                   | 15 |
| 3.2  | Traiettorie generate - $\delta_f = 2$ . . . . .                                                 | 16 |
| 3.3  | Distribuzione angolo di curvatura - $\delta_f = 2$ . . . . .                                    | 17 |
| 3.4  | Istogramma delle lunghezze in metri delle traiettorie - $\delta_f = 2$ . . . . .                | 17 |
| 3.5  | Distribuzione della distanza fra punti consecutivi delle traiettorie - $\delta_f = 2$ . . . . . | 18 |
| 3.6  | Framerate originali - $\delta_f = 2$ . . . . .                                                  | 19 |
| 3.7  | Distribuzione dei pixel del frame RGB coperti dalla point-cloud - $\delta_f = 2$ . . . . .      | 19 |
| 3.8  | Traiettorie generate - $\delta_f = 4$ . . . . .                                                 | 20 |
| 3.9  | Distribuzione angolo di curvatura - $\delta_f = 4$ . . . . .                                    | 21 |
| 3.10 | Istogramma delle lunghezze in metri delle traiettorie - $\delta_f = 4$ . . . . .                | 21 |
| 3.11 | Traiettorie generate - $\delta_f = 6$ . . . . .                                                 | 22 |
| 3.12 | Distribuzione angolo di curvatura - $\delta_f = 6$ . . . . .                                    | 23 |
| 3.13 | Istogramma delle lunghezze in metri delle traiettorie - $\delta_f = 6$ . . . . .                | 23 |

# Introduzione

Nel contesto della guida autonoma, le reti neurali artificiali acquisiscono un ruolo chiave in quanto consentono al veicolo di elaborare e interpretare in tempo reale le informazioni provenienti da vari sensori; queste informazioni permettono di ricostruire l'ambiente in cui si trova, riconoscendo i vari elementi della strada (come altri veicoli, pedoni, ostacoli o segnali stradali) per poi decidere le manovre da effettuare.

Affinché una rete neurale sviluppi la precisione e l'affidabilità necessarie per poter introdurre in strada il veicolo autonomo, è necessario che questa venga addestrata sulla base di una grande quantità di dati.

La creazione di un dataset ben strutturato richiede però un grande sforzo nella raccolta di molti dati reali e diversificati; l'addestramento su un dataset sbilanciato può infatti portare la rete neurale a comportarsi bene in alcuni contesti specifici ma a fallire totalmente in altri.

Questa tesi si pone il problema di creare un dataset adatto al corretto addestramento di moduli di guida autonoma, nello specifico nella generazione di traiettorie future. I dati sono stati raccolti utilizzando un veicolo radiocomandato denominato come *modello AutoRC*

Il presente documento è strutturato in tre capitoli:

- Il primo capitolo è dedicato alla descrizione delle caratteristiche software e hardware del modello AutoRC, con particolare attenzione al metodo salvataggio dei dati registrati.
- Il secondo capitolo si concentra sulla procedura di acquisizione dati e sul funzionamento del modulo software che li elabora.
- Nel terzo, infine, si presenta l'analisi di alcuni possibili dataset ottenibili a partire dai dati acquisiti, soffermandosi su alcune statistiche significative.

# Capitolo 1

## Descrizione del modello AutoRC

Il modello AutoRC è il frutto del lavoro coordinato fra il laboratorio di sistemi di controllo e quello di elettronica dell'Università degli Studi di Firenze, che si sono occupati di modificare una macchina elettrica telecomandata applicandovi della componentistica, fra cui sensori, microprocessori e microcontrollori, con l'obiettivo di creare un ambiente che facilitasse lo sviluppo e il testing di algoritmi per la guida autonoma.

### 1.1 Specifiche del modello e dispositivi equipaggiati

La macchina elettrica telecomandata è una Traxxas E-Revo 70+MPH e le sue dimensioni sono in scala 1:8 rispetto a quelle reali; precisamente è lunga 585 mm e larga 465 mm.

Pesa circa 5 kg ma arriva a superare gli 8 Kg una volta montata tutta la componentistica

La macchina è alimentata da due batterie LiPo EZp 4000/2-TXX, ciascuna con 4000 mAh e tensione nominale di 7,4 V.



Figura 1.1: Il modello AutoRC

I dispositivi montati sulla macchina necessari al suo controllo e per permettere la guida autonoma sono:

- Teensy 4.1 Development Board: ricopre il ruolo di MUX (Multiplexer);
- Sensore ottico Pixart PMW3389DM-T3QU: montato su ciascuna ruota del veicolo e utile per misurare la rotazione di queste;
- Arduino Due: svolge il compito di controllore del moto della macchina;
- Scheda Nucleo\_L432KC: riceve ed elabora i dati forniti dai sensori ottici per poi trasmetterli al blocco MUX;
- Scheda IMU: presenta accelerometri e giroscopi utili per determinare l'assetto del veicolo;
- NVIDIA Jetson AGX Orin;
- Lidar;
- Telecamera stereoscopica ZED 2i.

Entriamo più nello specifico per quanto riguarda la telecamera ZED e le sue funzionalità, fondamentali per lo sviluppo del progetto.

### 1.1.1 Telecamera ZED Stereolabs 2i

La telecamera stereo ZED 2i presenta due lenti distanti 12 cm che, simulando il funzionamento della visione binoculare umana, consentono di catturare video 3D ad alta risoluzione della scena da due diversi punti di vista e stimare la profondità e il movimento confrontando lo spostamento dei pixel tra le immagini sinistra e destra.

Le mappe di profondità (*depth*) acquisite dalla telecamera registrano un valore di distanza (*Z*) per ogni pixel (*X*, *Y*) dell'immagine; questa distanza, espressa in unità metriche, viene calcolata a partire dalla parte posteriore dell'occhio sinistro della telecamera all'oggetto nella scena. Per visualizzare le mappe, essendo queste codificate su 32 bit, è necessaria una rappresentazione monocromatica (in scala di grigi) a 8 bit con valori compresi nell'intervallo  $[0, 255]$ , dove 255 rappresenta il valore di profondità più vicino possibile e 0 il valore più distante.

Inoltre la ZED 2i è dotata di un'IMU (Inertial Measurement Unit) che misura e riporta la sua posizione e il suo orientamento, utilizzando una combinazione di accelerometri e giroscopi.

La sincronizzazione e combinazione dei dati visivi e inerziali consente quindi di ottenere una comprensione dettagliata del movimento e della struttura spaziale in cui esso avviene.

Un altro modo per rappresentare le informazioni di profondità è mediante una nuvola di punti (*point-cloud*). Una nuvola di punti può essere vista come una mappa di profondità in tre dimensioni: se la *depth* contiene solo l'informazione della distanza (*Z*) per ogni pixel, una *point-cloud* è una raccolta di punti 3D (*X*, *Y*, *Z*) che rappresentano la superficie esterna della scena. Al fine di rendere più comprensibile la rappresentazione dell'ambiente, i vari punti che compongono la *point-cloud* possono essere colorati con il colore del pixel corrispondente nel frame RGB.

## 1.2 Infrastruttura del sistema

L'infrastruttura di tutto il sistema può essere rappresentata tramite uno schema a blocchi (Figura 1.2), ognuno dei quali ha un ruolo specifico nella gestione delle funzionalità del sistema.

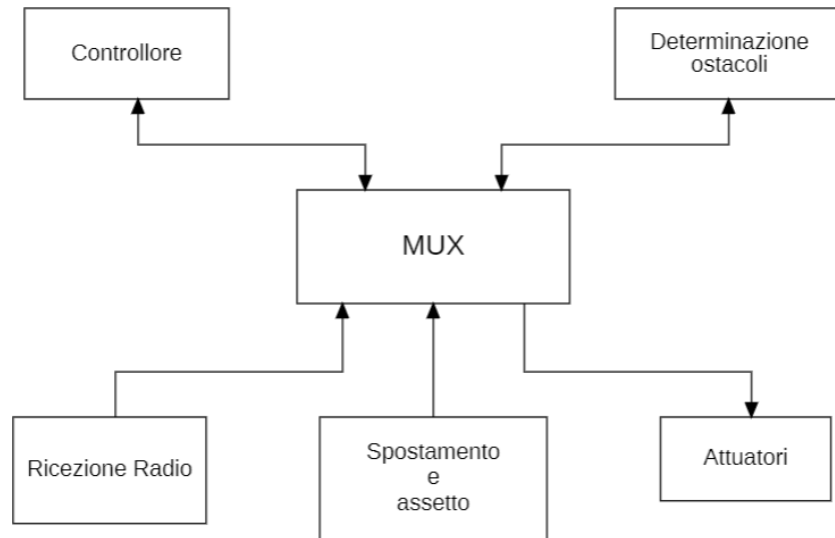


Figura 1.2: Schema a blocchi rappresentante l'infrastruttura del sistema

### 1.2.1 Blocco MUX

Questo blocco è il componente principale del sistema ed è rappresentato unicamente dalla scheda Teensy. Si occupa di ricevere i dati relativi allo spostamento, all'assetto e alla distanza degli ostacoli (opportunamente pre-elaborati) e successivamente di reindirizzarli al sistema di controllo dell'auto, determinando quale azione compiere in base alla modalità di guida selezionata.

Nella modalità di "guida autonoma" il MUX passa agli attuatori i dati ricevuti dal controllore autonomo, mentre, se l'auto si trova in modalità di "controllo da remoto", invia loro quelli provenienti dal blocco di ricezione radio.

Potenzialmente è in grado di mettere in comunicazione i due blocchi, permettendo una modalità ibrida in cui viene effettuato un controllo di trazione anche quando la macchina è guidata da un utente.

### 1.2.2 Blocco controllore

Il blocco controllore è rappresentato dalla scheda Arduino Due che è responsabile del controllo del movimento dell'auto quando questa si trova nella modalità di "guida autonoma". Si occupa di eseguire algoritmi di controllo che gestiscono trazione e direzione del veicolo in base alle informazioni ricevute dagli altri blocchi del sistema.



### 1.2.3 Blocco spostamento e assetto

Questo è il blocco che comprende il maggior numero di componenti: la scheda Nucleo 32, la scheda IMU e i quattro sensori ottici. Ha il compito di raccogliere ed elaborare i dati forniti dagli accelerometri e giroscopi della IMU insieme a quelli sul movimento delle ruote, necessari alla stima dell'assetto del veicolo e della sua velocità.

Sulla scheda Nucleo 32 è stato sviluppato un firmware che gli consente di interpretare i dati provenienti dai sensori, con i quali comunica tramite il protocollo SPI (Serial Peripheral Interface).

I sensori rilevano gli spostamenti delle ruote sugli assi X e Y, determinandone anche il verso; possiedono anche un'unità di calcolo integrata che fornisce direttamente l'entità dello spostamento. A differenza dei sistemi realizzati con fototransistor ed encoder, che richiedono la presenza di due transistor per determinare la direzione di rotazione, l'utilizzo di questi sensori ottici offre una soluzione più compatta e integrata. I sensori sono posizionati all'interno dei cerchioni e osservano una superficie piana realizzata tramite stampa 3D.

## 1.3 Salvataggio messaggi attraverso il framework ROS2

ROS2 (Robot Operating System 2) è un framework open-source utilizzato per lo sviluppo software per robot. Alla base di ogni sistema ROS2 vi è un grafo ROS, una rete di nodi ROS capaci di comunicare tra loro tramite una libreria client ROS.

In ROS2, un nodo è un elemento fondamentale che svolge uno scopo singolo e modulare in un sistema robotico; nella pratica, è un processo autonomo espresso come file eseguibile all'interno di un ROS package.

Con *topic* in ROS2 si intende un canale di comunicazione utilizzato dai nodi per lo scambio di dati, organizzati in *messaggi*.

Ogni messaggio è strutturato secondo un tipo predefinito che ne specifica il formato e i dati contenuti al suo interno. Per definire un tipo di messaggio si utilizza il formato `.msg`, nativo di ROS.

I nodi hanno la possibilità di pubblicare messaggi sui topic o di iscriversi ad essi per riceverne. Altre modalità di comunicazione fra nodi possono essere *servizi* e *azioni*.

Mentre i topic adottano un sistema di publisher-subscriber, i servizi si basano su un modello di richiesta e risposta in cui un client fa una richiesta al nodo che fornisce il servizio, il quale elabora la richiesta e genera una singola risposta. In generale è quindi opportuno evitare di ricorrere a questi per chiamate continue.

Le azioni sono simili ai servizi, con la differenza che le prime forniscono un feedback continuo, rendendole quindi più adatte per compiti a lungo termine. Le azioni seguono un modello client-server in cui un nodo "client di azione" invia un obiettivo a un nodo "server di azione" che riconosce l'obiettivo e restituisce un flusso di feedback e un risultato.

Il modello AutoRC sfrutta in particolar modo la comunicazione tramite topic registrando e salvando i messaggi pubblicati dai nodi grazie alle funzionalità offerte da *rosbag*.

Rosbag è uno strumento di ROS2 che permette di registrare e riprodurre messaggi inviati

tra i nodi. Nello specifico, il comando `ros2 bag record` consente la registrazione di tutti i messaggi scambiati su uno o più topic indicati.

Il loro contenuto (assieme al timestamp associato) viene salvato in un file con l'estensione `.bag`, che è di fatto una cartella contenente:

- Un file `.db3` che memorizza l'effettivo contenuto dei messaggi.
- Un file `metadata.yaml` che include dettagli sulla registrazione come durata, topic coinvolti e tipi di messaggi salvati.

Il comando `ros2 bag play` permette inoltre di riprodurre i messaggi registrati come se fossero nuovamente pubblicati dai nodi, ripetendo quindi le stesse condizioni di input.

L'utilizzo di `rosbag` consente quindi di salvare una grande quantità di dati sincronizzati registrati dai sensori durante una sessione dal vivo per poterli poi analizzare offline tramite la riproduzione col comando sopra citato o direttamente estraendoli dal file `.db3`.

La registrazione può essere avviata manualmente o automatizzata tramite script ROS2 che iniziano la registrazione non appena vengono rilevati dati disponibili sui topic. Nel caso del modello AutoRC la registrazione viene avviata tramite l'attivazione di un'apposita levetta sul radiocomando.

Ogni componente del sistema (telecamera, Lidar, radiocomando) pubblica i propri dati su un topic specifico; ad esempio, la ZED StereoLabs 2i pubblica le immagini stereo e i dati di profondità su un topic dedicato, mentre il Lidar invia le sue letture delle distanze su un altro topic.

I comandi di guida, la telemetria e altri segnali provenienti dai sensori sono gestiti da nodi ROS2 chiamati *wrapper*; questi hanno il compito di intercettare i segnali del radiocomando, che sono solitamente codificati in PWM (*Pulse Width Modulation*), e di convertirli in un formato compatibile con ROS2.

Il PWM è una tecnica che permette di codificare segnali analogici utilizzando un segnale digitale, modificando la durata degli impulsi all'interno di un ciclo. Una volta convertiti, i wrapper si occupano di pubblicare questi comandi su topic specifici, rendendoli accessibili agli altri nodi del sistema.

## Capitolo 2

# Raccolta ed elaborazione dei dati

In questo capitolo sarà trattata la procedura di acquisizione dei dati col modello AutoRC e la formazione del dataset fatta a partire da questi ultimi.

### 2.1 Procedura di acquisizione dei dati

Il modello AutoRC è custodito all'interno del laboratorio SysCon (Systems and Control Lab) situato nella sede della Scuola di Ingegneria di Santa Marta.

Il procedimento di raccolta dei dati inizia con la verifica dello stato di carica delle batterie attraverso il relativo caricabatterie (modello Traxxas EZ Peak Live Dual) ed eventualmente con la loro ricarica.

Successivamente è necessario accendere la Jetson Orin (alimentata dalle batterie) e avviare i nodi Ros2 relativi al radiocomando, al sensore Lidar e alla telecamera Zed. Per farlo è possibile collegarsi alla seriale tramite cavo USB-C oppure direttamente dalla Jetson connettendo uno schermo e una tastiera.

Le decisioni riguardo alla modalità di registrazioni, quali luogo e manovre da eseguire, sono conseguenti alla necessità di simulare una guida di un veicolo di medie dimensioni lungo una qualsiasi strada urbana.

Per questa ragione le sessioni di guida sono state registrate di fronte all'ingresso principale di Santa Marta, il cui cortile è zona carrabile e presenta caratteristiche compatibili con le esigenze sopra citate: veicoli parcheggiati ai lati della carreggiata, varie curve, un incrocio a quattro e uno a T.

Le singole sequenze di registrazione, avviate e interrotte tramite un'apposita levetta sul radiocomando, comprendono di norma l'esecuzione di una curva o l'approccio a un incrocio (con conseguente riduzione della velocità in prossimità di quest'ultimo).

Talvolta la registrazione viene avviata con l'auto posizionata in modo tale da rendere necessario l'utilizzo di manovre di assestamento per riportarla al centro della carreggiata; questo accorgimento serve a rendere il dataset più vario tramite l'inclusione di traiettorie più realistiche. Al termine di ogni sessione i dati vengono salvati in attesa di essere processati.

## 2.2 Estrazione dei campioni dalle registrazioni

I dati acquisiti nelle varie sessioni di registrazione sono stati elaborati per ricavarne una serie di campioni da dare in ingresso alla rete neurale.

A questo scopo è stato realizzato un modulo software in linguaggio Python consultabile sulla piattaforma GitHub al seguente indirizzo: <https://github.com/Tacce/AutoRC>.

### 2.2.1 Struttura delle classi del modulo software

Analizziamo adesso le principali classi che compongono il modulo software, evidenziandone le responsabilità individuali e il modo in cui interagiscono fra loro.

Il diagramma UML riportato in Figura 2.1 offre una rappresentazione visiva della struttura del sistema e delle relazioni tra le classi.

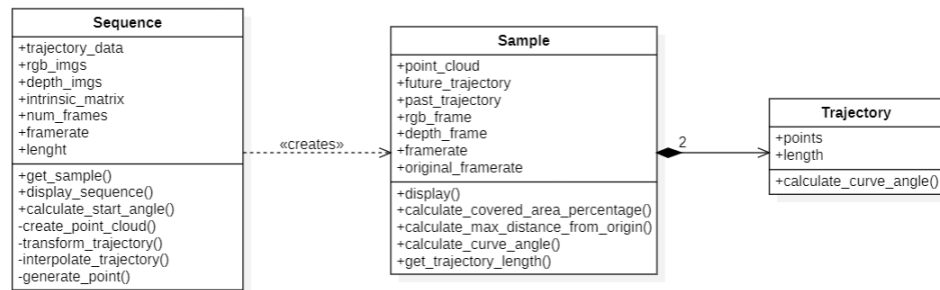


Figura 2.1: Diagramma delle classi del modulo software

Le singole sequenze di registrazione sono rappresentate dalla classe *Sequence* che è in primo luogo responsabile di estrarre le informazioni relative all’odometria e ai frame RGB e *depth* dalle cartelle rosbag2. Nello specifico i topic ros2 interessati sono:

- `/zed/zed_node/odom` per la posizione e l’orientamento
- `/zed/zed_node/right/image_rect_color` per i frame RGB
- `/zed/zed_node/depth/depth_registered` per le mappe di profondità.

Fra gli attributi di questa classe troviamo altre informazioni sulla sequenza come la lunghezza espressa in secondi e in numero di frame, oltre alla matrice intrinseca della fotocamera ZED, fondamentale per il processo di proiezione della scena 3D sul sensore 2D della fotocamera.

La classe *Sample* rappresenta invece un campione individuale generato da una sequenza e include una nuvola di punti, le traiettorie passata e futura assieme ai frame RGB e *depth* associati.

I metodi di questa classe hanno lo scopo di estrarre informazioni sul singolo campione, le quali possono essere utilizzate nell’analisi complessiva delle statistiche del dataset.

La traiettoria futura e quella passata sono rappresentate come oggetti della classe *Trajectory*.

### 2.2.2 Il metodo *get\_sample()*

Il metodo *get\_sample()* della classe *Sequence* rappresenta la funzione più rilevante del codice poiché consente di generare vari campioni (rappresentati come istanze della classe *Sample*) a partire da una singola sequenza.

Tale metodo è definito come segue:

```
get_sample(t, delta_f, delta_p, framerate, max_velocity)
```

Dato in ingresso un frame *t*, la funzione restituisce un campione le cui traiettorie, futura e passata, sono definite entro gli intervalli *delta\_f* e *delta\_p* (anch'essi forniti in ingresso ed espressi in secondi) a partire dal timestamp del *t*-esimo frame.

A causa della natura discreta dei dati raccolti, quando si vanno a considerare intervalli continui come quelli sopra citati, è molto probabile che i timestamp dei dati non si allineino esattamente con i loro estremi. Una soluzione a questo problema potrebbe essere quella di scegliere come ultimo punto della traiettoria quello immediatamente precedente o successivo alla soglia temporale richiesta, ma questo significherebbe ottenere traiettorie molto variabili in lunghezza, oltre a imprecisioni sulla stima di velocità e accelerazione del veicolo. Si è quindi deciso di ricorrere all'utilizzo di una subroutine, *generate\_point()*, che si occupa di stimare la posizione della macchina dato un timestamp preciso.

La funzione sfrutta l'interpolazione lineare fra l'ultima posizione registrata prima del timestamp scelto e la prima dopo di esso e calcola il fattore di interpolazione *k*, che rappresenta la frazione della distanza tra i due punti, basato sul timestamp desiderato; il valore ottenuto risulta compreso fra 0 e 1 e indica quanto il timestamp desiderato è vicino al punto successivo rispetto al quello precedente.

$$k = (\text{timestamp} - \text{last\_time}) / (\text{next\_time} - \text{last\_time})$$

Il fattore di interpolazione *k* viene poi utilizzato per la stima della posizione come segue:

$$\text{InterpolatedPosition} = \text{last\_pos} + k * (\text{next\_pos} - \text{last\_pos})$$

La funzione *get\_sample()* prende in ingresso anche il *framerate* richiesto e, nel caso in cui il numero di punti non sia quello atteso, ne vengono aggiunti altri tramite interpolazione quadratica. Questo controllo viene effettuato sia sulla traiettoria futura che su quella passata. Durante la generazione del sample è possibile che una determinata combinazione di parametri non consenta di generare un campione valido: in questi casi è compito della funzione *get\_sample()* segnalarlo restituendo il valore *None*. Ciò accade nelle seguenti circostanze:

1. Il timestamp del frame d'interesse è troppo vicino all'inizio o alla fine della traiettoria e quindi non ci sono abbastanza dati sia per coprire l'intervallo *delta\_p* nel passato o *delta\_f* nel futuro, sia per generare con precisione il punto interpolato agli estremi.
2. Le traiettoria calcolata nell'intervallo *delta\_f* e *delta\_p* contengono meno di 3 punti e quindi non è possibile interpolare quadraticamente.

3. La distanza fra punti consecutivi della traiettoria (prima dell'interpolazione) risulta sproporzionata rispetto all'intervallo di tempo fra un'acquisizione della posizione e l'altra. A questo proposito `get_sample()` riceve in ingresso un parametro `max_velocity` che funge da upper-bound per la velocità del veicolo.

Il controllo su quest'ultimo punto permette di escludere campioni che presentano un numero insufficiente di punti a causa della mancata acquisizione di dati odometrici per alcuni istanti.

Il metodo `get_sample()`, infine, sfruttando i timestamp dei messaggi, si occupa di trovare l'immagine RGB e la *depth* catturate nell'istante più vicino a quello in cui è stata registrata la *t*-esima posizione e le usa per generare una nuvola di punti (definita nel precedente capitolo nella sezione 1.1.1).

Ai fini dell'addestramento della rete neurale, il sistema di riferimento della *point-cloud* e delle traiettorie deve essere lo stesso perciò vengono effettuate delle rototraslazioni su queste ultime, basandosi sull'orientamento della fotocamera ZED, che le trasportano nel sistema di riferimento della nuvola di punti (Figura 2.2). In seguito a questa operazione, il primo punto della traiettoria coincide con l'origine degli assi.

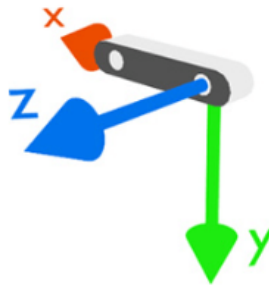


Figura 2.2: Sistema di riferimento della fotocamera ZED

Per generare il dataset vero e proprio è quindi necessario fissare i parametri `delta_f`, `delta_p`, `framerate` e `max_velocity` e chiamare `get_sample()` iterando su tutti i frame (e quindi sul parametro *t*).

I campioni validi, le cui informazioni sono espresse come `np.array`, vengono poi numerati e salvati come dizionari in un file `.npz`, in modo tale che in fase di addestramento sia possibile caricare esclusivamente i dati necessari.

## 2.3 Visualizzazione dei campioni

Una volta generato un campione valido tramite la chiamata a `get_sample()`, la funzione `display()` consente visualizzare la nuvola di punti, la traiettoria futura e quella passata nello stesso sistema di riferimento tridimensionale sfruttando le funzionalità offerte dalla libreria *Open3D*.

Di seguito sono riportati alcuni esempi.

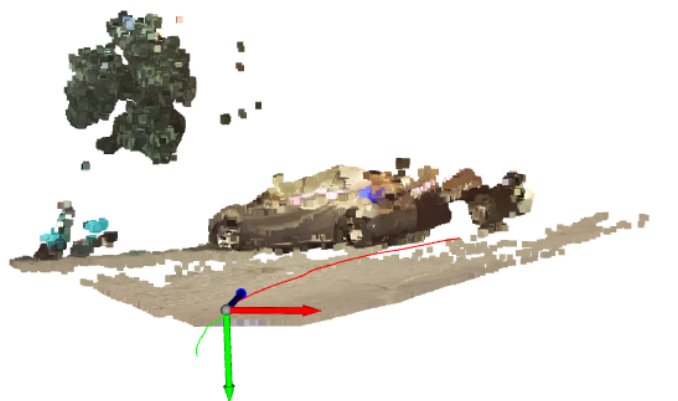


Figura 2.3: Visualizzazione della nuvola di punti e della traiettoria- Campione 1

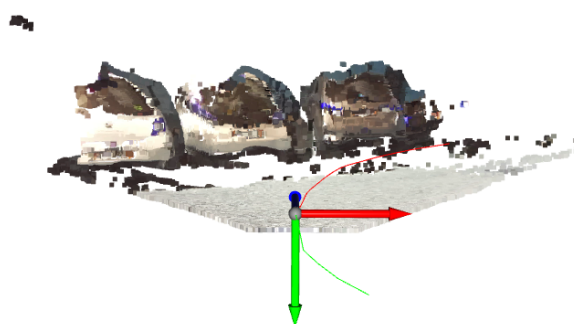


Figura 2.4: Visualizzazione della nuvola di punti e della traiettoria - Campione 2

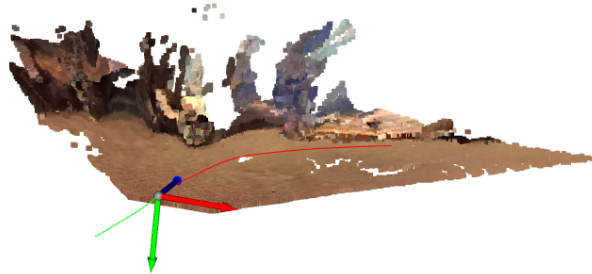


Figura 2.5: Visualizzazione della nuvola di punti e della traiettoria - Campione 3



Figura 2.6: Visualizzazione della nuvola di punti e della traiettoria - Campione 4





Figura 2.7: Visualizzazione della nuvola di punti e della traiettoria - Campione 5

Al fine di comprendere meglio il percorso seguito dalla macchina, può essere utile cercare di ricostruire l'ambiente in cui questa si muove nel contesto di un'intera sequenza. La funzione *display\_sequence()* si occupa di visualizzare nello stesso sistema di riferimento il mosaico(?) di tutte le point-cloud, posizionate e orientate in base alla disposizione della videocamera nel momento di cattura del frame. Il sistema di riferimento è quello della nuvola di punti ricavata dal primo frame della sequenza.

Segue una serie di esempi.

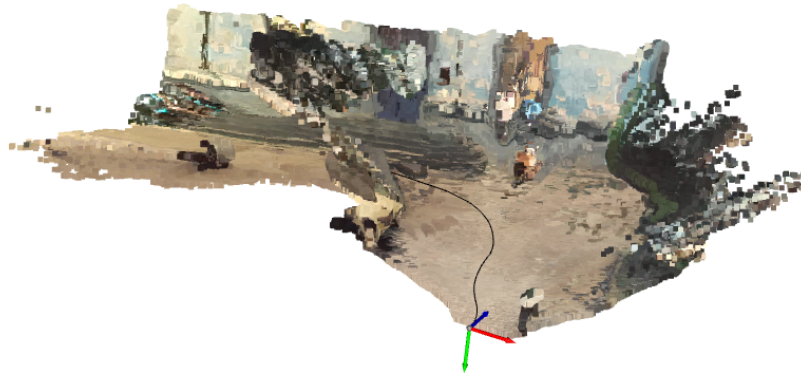


Figura 2.8: Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 1

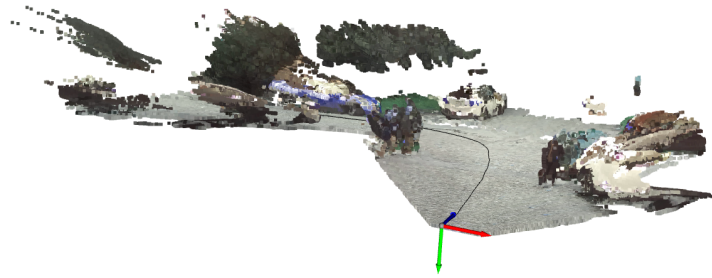


Figura 2.9: Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 2

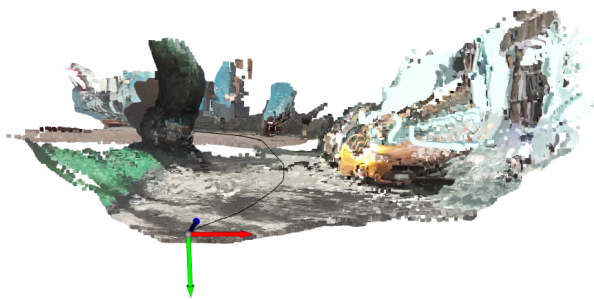


Figura 2.10: Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 3

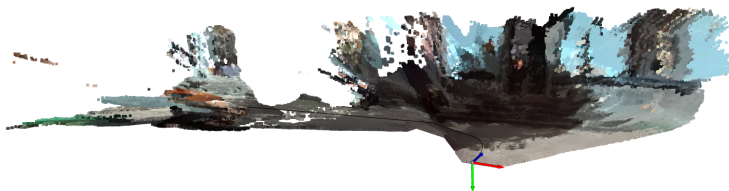


Figura 2.11: Ricostruzione dell'ambiente tramite mosaico di point-cloud - Sequenza 4

## Capitolo 3

# Analisi statistiche dei dati raccolti

In questo capitolo verranno analizzate statistiche riguardo diversi dataset ottenibili a partire dalle stesse acquisizioni variando alcuni parametri in ingresso al modulo software.

Al fine di comprendere meglio le analisi svolte sui dati è necessario considerare che le sequenze sono state acquisite con impostazioni differenti: in alcune i frame RGB e le *depth* sono state salvate con una risoluzione di 336x118 pixel, mentre altre presentano immagini di dimensione 640x360 pixel al costo di una riduzione del framerate.

### 3.1 Statistiche generali delle sequenze

In totale sono stati raccolti 4019 secondi effettivi di registrazione divisi in 199 sequenze. Nello specifico, in 73 sequenze i frame sono registrati a bassa risoluzione per una durata totale di 1524 s mentre le restanti 125 coprono 2495 s e presentano frame a qualità superiore. La durata media delle sequenze è di 20,2 secondi.

#### 3.1.1 Angolo di curvatura iniziale

Come accennato nel paragrafo riguardo la procedura di acquisizione, alcune sequenze iniziano con la macchina ai lati della carreggiata, perciò è interessante analizzare l'angolo di curvatura iniziale.

L'angolo considerato è quello formato dalla retta passante per i primi due punti della traiettoria rispetto all'asse Z (che punta in avanti rispetto all'orientamento della telecamera nel primo frame della sequenza).

Come si può notare in Figura 3.1, la maggior parte degli angoli iniziali si concentra in un intervallo ristretto intorno allo zero, indicando una forte tendenza a iniziare con traiettorie relativamente diritte o con deviazioni minime.

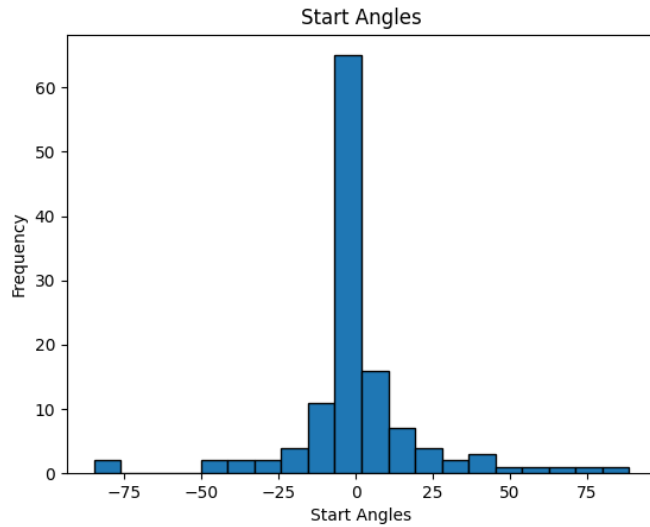


Figura 3.1: Distribuzione dell'angolo d'attacco

Nelle prossime sezioni si analizzano alcuni possibili dataset ottenibili dai dati registrati.

Dal momento che l'attenzione è rivolta principalmente alle traiettorie future dei singoli campioni, l'unico parametro che è stato modificato nelle varie versioni è  $\text{delta\_f}$ ; iterando su esso (oltre che su  $t$ ) sono state generate traiettorie dalla lunghezza di 2, 4 e 6 secondi.

Gli altri parametri sono impostati come segue:

- $\text{delta\_p} = 2$
- $\text{framerate} = 10$
- $\text{max\_velocity} = 2,8$

## 3.2 Dataset 1: $\text{delta\_f} = 2$

Utilizzando questa particolare assegnazione di parametri sono stati generati 10816 campioni.

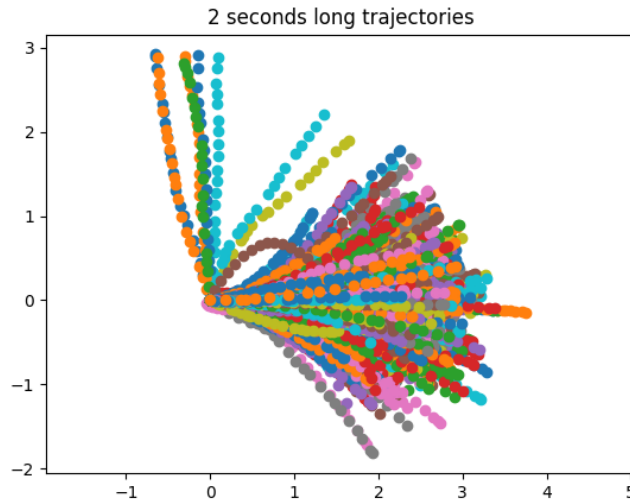
### 3.2.1 Trattorie del dataset

La figura 3.3 mostra sullo stesso grafico un campione delle traiettorie generate.

### 3.2.2 Angolo di curvatura

L'analisi della distribuzione degli angoli di curvatura delle traiettorie è utile in quanto permette di conoscere indicativamente quante sono le curve a destra, quante quelle a sinistra e quanto quelle sostanzialmente rettilinee. L'angolo di curvatura totale del campione viene determinato come differenza fra angolo finale e iniziale, calcolati rispettivamente guardando gli ultimi due e i primi due punti della traiettoria.

Esattamente come per l'angolo di curvatura iniziale delle traiettorie (sezione 3.1), anche qui

Figura 3.2: Traiettorie generate -  $\text{delta}_f = 2$ 

l'angolo è riferito rispetto all'asse Z: ciò significa che un angolo positivo indica una curva a destra, un angolo negativo una curva a sinistra e un angolo intorno allo 0 rappresenta una traiettoria pressoché dritta.

Il calcolo della curvatura totale della traiettoria viene effettuato solo nel caso in cui la macchina abbia le ruote dritte all'inizio del campione; nello specifico si procede con la stima solo se il valore assoluto della pendenza fra il primo e il secondo punto è minore di  $1/6$  (cioè se l'angolo iniziale è minore di circa  $9,5^\circ$ ).

Come mostrato in figura 3.3, la distribuzione ha una forma simile ad una campana, suggerendo che possa essere interpretata come una gaussiana. La maggior parte dei campioni presenta delle curve attorno agli  $0^\circ$ , con picco massimo intorno a 0, il che indica che le curve con angoli piccoli (o addirittura senza curva) sono più frequenti.

La media dell'angolo di curva è di  $1,547^\circ$ , il che indica una generale simmetria fra le curve a destra e quelle a sinistra. La varianza è invece 429,2.

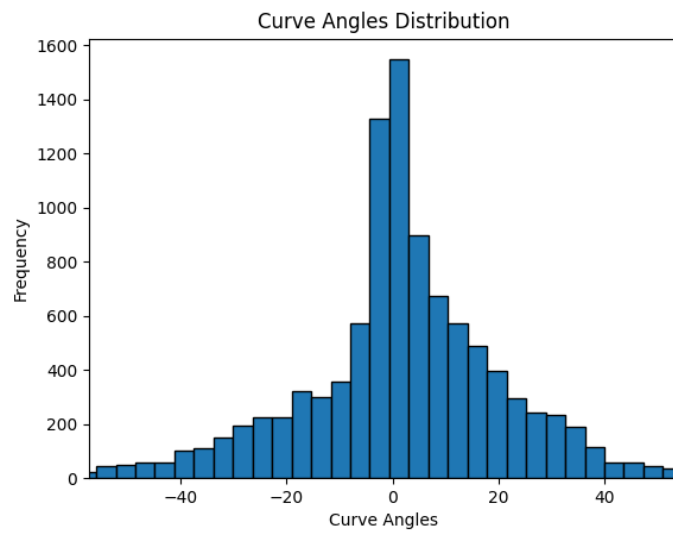
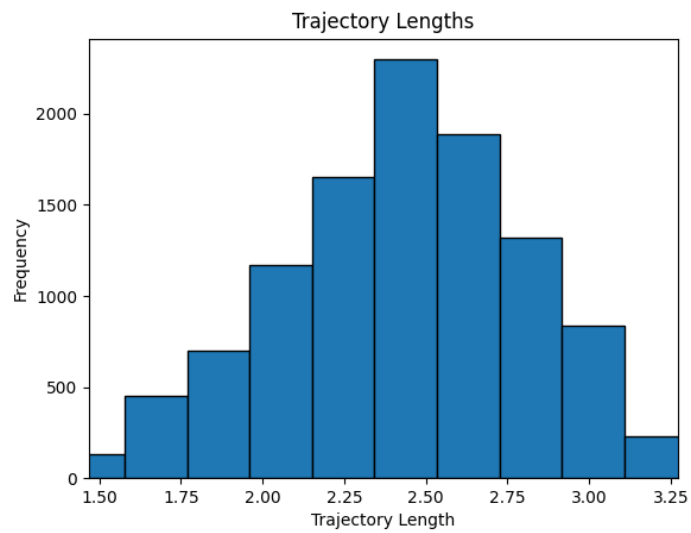
Come mostrato in figura 3.3, la distribuzione ha una forma simile ad una campana, suggerendo che possa essere interpretata come una gaussiana. La maggior parte dei campioni presenta delle curve attorno agli  $0^\circ$ , con picco massimo intorno a 0, il che indica che le curve con angoli piccoli (o addirittura senza curva) sono più frequenti.

La media dell'angolo di curva è di  $1,547^\circ$ , il che indica una generale simmetria fra le curve a destra e quelle a sinistra. La varianza è invece 429,2.

### 3.2.3 Lunghezza delle traiettorie

La figura 3.4 mostra la distribuzione della lunghezza delle traiettorie dei campioni estratti. Anche in questo caso la distribuzione assomiglia ad una normale sebbene in questo caso i valori siano concentrati in un intervallo più ristretto.

La lunghezza media è 2,429 m mentre la varianza è 0,152.

Figura 3.3: Distribuzione angolo di curvatura -  $\delta f = 2$ Figura 3.4: Istogramma delle lunghezze in metri delle traiettorie -  $\delta f = 2$

### 3.2.4 Distanza fra punti consecutivi delle traiettorie

Nella figura 3.5 si può osservare la distribuzione delle distanze fra punti consecutivi delle traiettorie future dei campioni generati.

La forma asimmetrica suggerisce che è più comune trovare distanze più grandi rispetto alla media (coda verso destra), anche se la loro frequenza diminuisce gradualmente.

La distanza media è 0,12 m e la varianza è 6,462. Il dato sulla media è coerente con quello della lunghezza delle traiettorie: avendo fissato  $framerate = 10$  e  $delta\_f = 2$  il numero di punti della traiettoria futura è sempre  $2 * 10 = 20$  per cui  $20 * 0,12 \text{ m} \approx 2,43 \text{ m}$ .

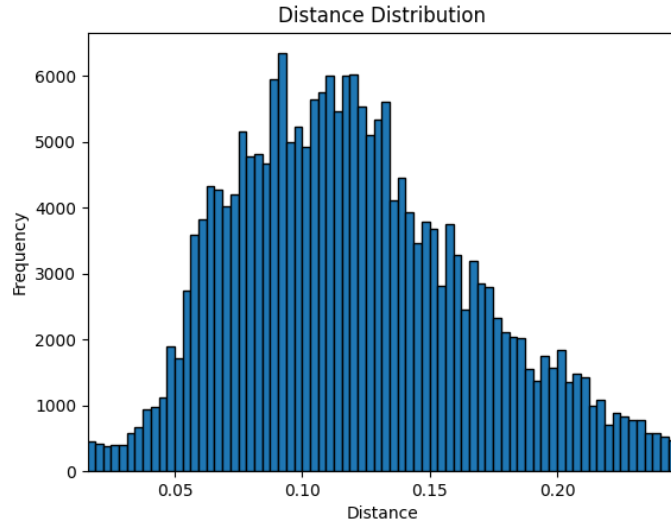


Figura 3.5: Distribuzione della distanza fra punti consecutivi delle traiettorie -  $delta\_f = 2$

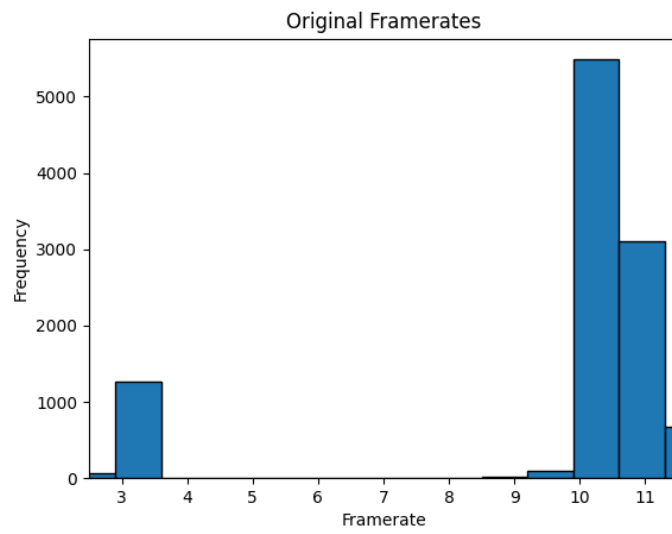
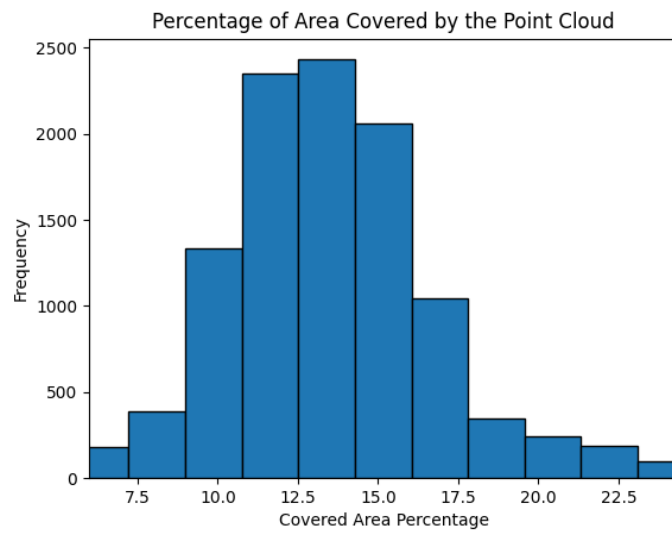
### 3.2.5 Framerate

Se osserviamo la distribuzione dei framerate originali dei singoli campioni (antecedenti cioè all'eventuale interpolazione) possiamo notare due picchi intorno ai 10,5 e i 3,5 frame al secondo: questi valori coincidono con i framerate delle sequenze rispettivamente a bassa e migliore qualità video. I campioni con framerate più alto risultano essere in numero maggiore dal momento che gli esempi che compongono il dataset sono generati a partire dai singoli frame e le sequenze a framerate maggiore ne contengono naturalmente di più. Il framerate medio è di 9,693 frame al secondo.

### 3.2.6 Area dell'immagine RGB coperta dalla point-cloud

L'area dell'immagine RGB coperta dalla point-cloud viene calcolata utilizzando il frame di profondità corrispondente: durante la fase di proiezione nello spazio 3D si determina quanti pixel sono stati proiettati correttamente per poi ricavarne la percentuale.

La distribuzione assume una forma simile a quella di una gaussiana con una media del 13.539% e varianza uguale a 11,770.

Figura 3.6: Framerate originali -  $\delta_f = 2$ Figura 3.7: Distribuzione dei pixel del frame RGB coperti dalla point-cloud -  $\delta_f = 2$



### 3.3 Dataset 2: $\text{delta}_f = 4$

Scegliendo un dataset con traiettorie più lunghe otteniamo un numero minore di campioni; nello specifico parliamo di 7268 elementi rispetto ai 10816 dell'istanza analizzata nel paragrafo precedente.

Per questa versione del dataset, come per quella successiva, non verranno analizzate le statistiche riguardo la distanza fra i punti consecutivi, la percentuale di immagine RGB coperta dalla nuvola di punti e il framerate originale dei campioni in quanto non presentano ulteriori spunti di riflessione rispetto a quelli già discussi per il Dataset 1.

#### 3.3.1 Trattorie del dataset

Facendo un confronto col caso precedente (Figura 3.2), le traiettorie generate con questa specifica combinazioni di parametri (Figura 3.8), presentano molti più punti (esattamente il doppio dato che il *framerate* non è stato modificato) e appaiono più disperse nello spazio.

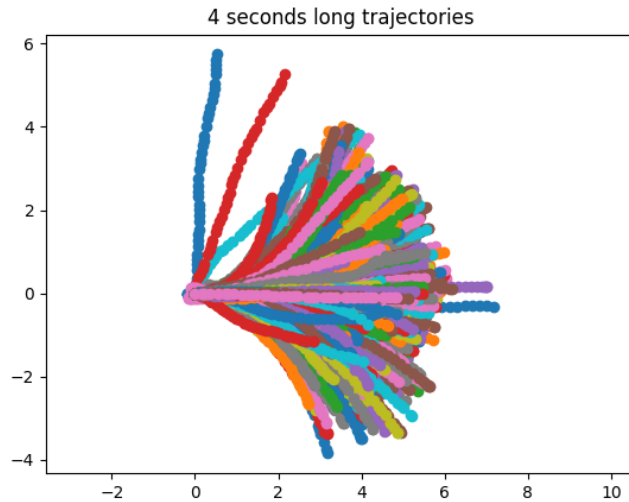


Figura 3.8: Traiettorie generate -  $\text{delta}_f = 4$

#### 3.3.2 Angolo di curvatura

Se affianchiamo le figure 3.3 e 3.9 notiamo che nel secondo dataset la distribuzione dell'angolo di curva appare più "frastagliata", con più picchi locali, e che si estende in un range più alto. L'aumento dell'intervallo di tempo nel quale si registra la traiettoria futura permette di generare curve che si chiudono ad angoli maggiori.

L'angolo medio di curva è uguale a  $2,606^\circ$ , mentre la varianza è ragionevolmente aumentata a 1229,7.

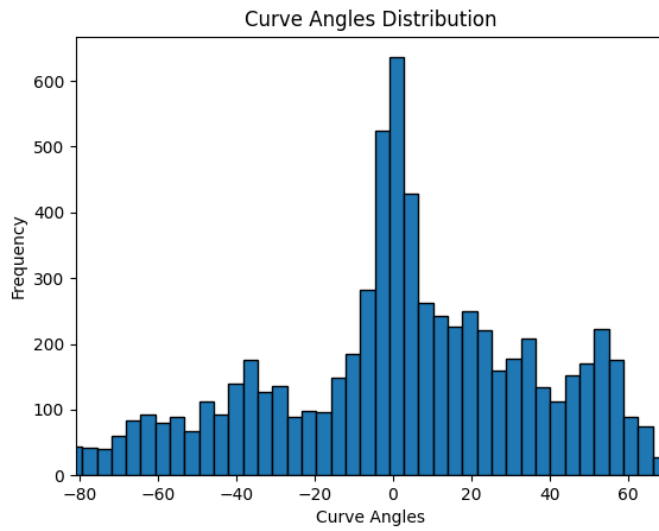


Figura 3.9: Distribuzione angolo di curvatura -  $\delta_f = 4$

### 3.3.3 Lunghezza delle traiettorie

Guardando la distribuzione della lunghezza delle traiettorie (Figura 3.10) si nota che la sua forma non presenta particolari differenze con il caso  $\delta_f = 2$ , tranne per il fatto che, trattando traiettorie più lunghe, il picco adesso si trova intorno ai 5,0 m; la lunghezza media delle traiettorie è infatti 4,841 m. Si osserva un incremento della varianza al valore 0,437.

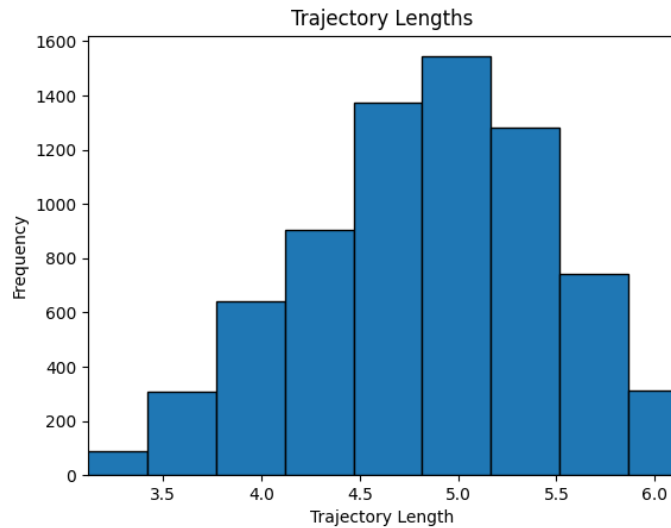


Figura 3.10: Istogramma delle lunghezze in metri delle traiettorie -  $\delta_f = 4$

### 3.4 Dataset 3: $\text{delta}_f = 6$

Aumentando ulteriormente la durata delle traiettorie a 6 s, il numero di campioni generati scende a 4924.

#### 3.4.1 Trattorie del dataset

Osservando la Figura 3.11 notiamo che rispetto ai Dataset 1 e 2, la dispersione delle traiettorie è ancora più marcata; queste adesso hanno ancora più tempo per divergere e allontanarsi maggiormente dal punto di origine.

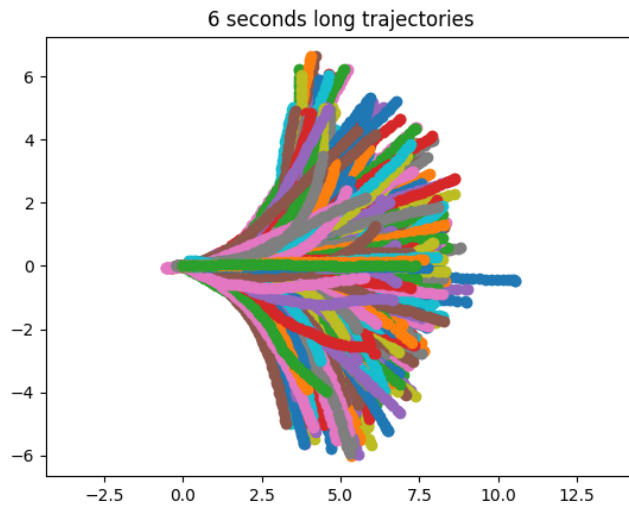


Figura 3.11: Traiettorie generate -  $\text{delta}_f = 6$

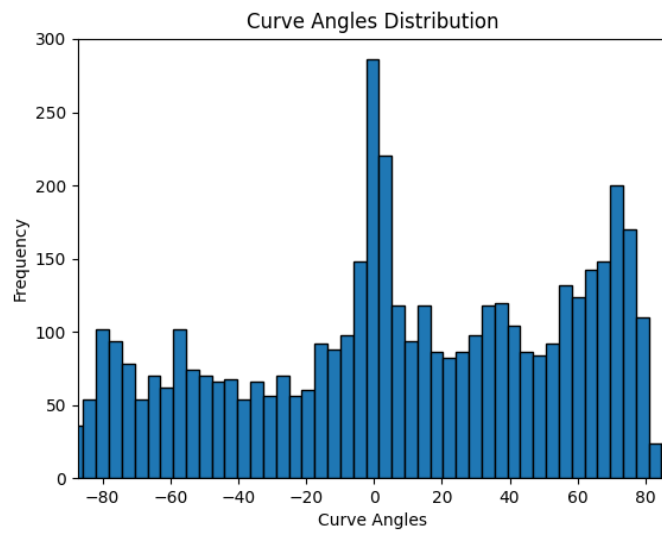
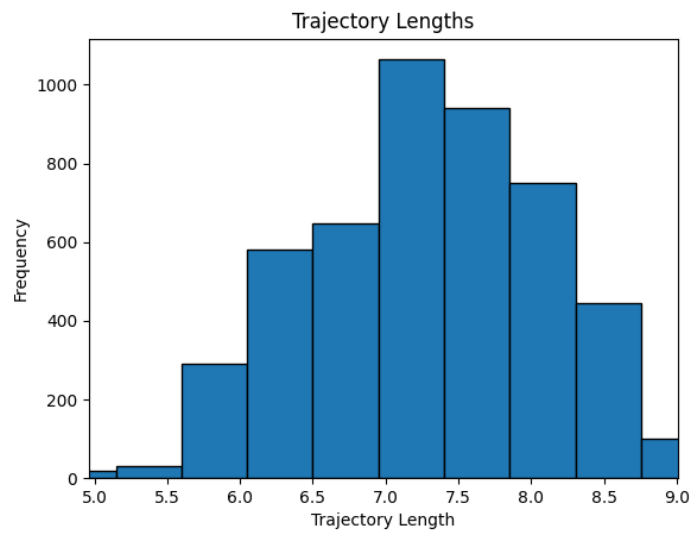
#### 3.4.2 Angolo di curvatura

La distribuzione dell'angolo di curva (Figura 3.12) riflette le possibilità di maggiore variazioni di direzione che si possono verificare in un intervallo di tempo più lungo. A fronte di un picco intorno agli  $0^\circ$ , che evidenzia come le traiettorie rettilinee rimangano le più frequenti, la distribuzione si mostra nel complesso più uniforme e più ampia rispetto ai casi precedentemente analizzati.

L'angolo medio di curvatura è  $8,097^\circ$ , ciò significa che questo particolare dataset presenta una lieve asimmetria verso le curve a destra. La varianza è aumentata a 2392,4.

#### 3.4.3 Lunghezza delle traiettorie

Anche con questa nuova configurazione, la distribuzione delle lunghezze delle traiettorie presenta una forma simile ad una campana. Le traiettorie hanno una lunghezza media di 7,272 m e la varianza è 0,748 (aumentando ulteriormente rispetto ai casi precedenti).

Figura 3.12: Distribuzione angolo di curvatura -  $\delta_f = 6$ Figura 3.13: Istogramma delle lunghezze in metri delle traiettorie -  $\delta_f = 6$

# Conclusioni

Lo sviluppo di questo progetto ha portato alla raccolta di molte sequenze di guida tramite l'utilizzo del modello *AutoRC*, un veicolo radiocomandato in scala 1:8.

È stato scelto un ambiente che potesse rappresentare al meglio le caratteristiche principali di una strada urbana, quali incroci, curve e divisione della carreggiata, e, allo stesso tempo, permettere la sicura circolazione del veicolo utilizzato per le riprese.

Infine è stato realizzato uno script che, partendo dalle sequenze registrate, estrae un dataset di vari campioni composti da una nuvola di punti, la traiettoria passata e quella futura (entro un intervallo di tempo definito).

Tra le opportunità di avanzamento del lavoro troviamo innanzitutto l'ampliamento del dataset attraverso nuove registrazioni in contesti diversi da l'unico preso in considerazione fino ad ora; la varietà del dataset è una caratteristica fondamentale per permettere alle reti neurali addestrate su di esso di essere affidabili in contesti reali. A questo proposito potrebbe essere utile acquisire dati anche in orari e condizioni atmosferiche differenti (anche se bisogna considerare che il modello *AutoRC* sia sensibile all'acqua e quindi alla pioggia).

Il dataset potrebbe essere quindi utilizzando per addestrare un modulo di guida autonoma per la generazione di traiettorie di un veicolo; dal momento che i dati registrati contengono anche le informazioni riguardo il radiocomando (nello specifico l'inclinazione delle leve di controllo), si potrebbe implementare un'effettiva guida autonoma del veicolo associando ad ogni traiettoria la serie di comandi per eseguirla.

# Bibliografia

- [1] Francesco Morini. Sviluppo hardware e software di una piattaforma robotica mobile per la sperimentazione di sistemi a guida autonoma. Tesi di laurea magistrale, Università degli studi di Firenze, 2021-2022.
- [2] Riccardo Vanni. Analisi, modellazione e controllo di un prototipo in scala di veicolo a guida autonoma. Tesi di laurea magistrale, Università degli studi di Firenze, 2022-2023.
- [3] Open Robotics. Ros 2 foxy documentation. <https://docs.ros.org/en/foxy/index.html>.
- [4] Stereolabs. Zed 2i stereo camera. <https://www.stereolabs.com/en-it/store/products/zed-2i>.
- [5] Stereolabs. Depth sensing documentation. <https://www.stereolabs.com/docs/depth-sensing>.